

Visualization, Charts and Visuals for Engineering Decisions in Python

Data visualization is the process of representing engineering data using graphs, charts, and plots so that engineers can understand patterns, compare alternatives, identify problems, and make better decisions.

1. Why visualization is important in engineering

Visualization helps engineers to:

- Analyze large amounts of data quickly.
- Identify trends and patterns.
- Detect errors, outliers, and abnormal values.
- Compare different designs, materials, machines, or processes.
- Monitor system performance.
- Predict future behavior.
- Communicate technical results clearly.
- Support evidence-based engineering decisions.
- Reduce the time required to interpret complex data.
- Improve safety, efficiency, and reliability.

For example, instead of examining thousands of temperature readings in a spreadsheet, an engineer can use a **line chart** to immediately identify when the temperature starts increasing abnormally.

2. Common charts used in Python

Chart	Purpose	Python Library
Line chart	Analyze trends over time	Matplotlib

Chart	Purpose	Python Library
Bar chart	Compare categories	Matplotlib
Pie chart	Show proportions	Matplotlib
Scatter plot	Find relationships between variables	Matplotlib
Histogram	Study data distribution	Matplotlib
Box plot	Detect outliers and compare distributions	Matplotlib / Seaborn
Heatmap	Show correlation or intensity	Seaborn
3D plot	Visualize three-dimensional data	Matplotlib
Area chart	Show cumulative changes over time	Matplotlib
Interactive chart	Explore data dynamically	Plotly

3. Example: Engineering performance comparison

Suppose an engineer wants to compare the performance of four machines based on their output.

For example:

- Machine A → 85 units/hour
- Machine B → 110 units/hour
- Machine C → 95 units/hour
- Machine D → 125 units/hour

A **bar chart** is suitable because the engineer needs to compare separate categories.

From the chart, Machine D has the highest output. This information can help the engineer select the most productive machine.

```
import matplotlib.pyplot as plt

machines = ["Machine A", "Machine B", "Machine C", "Machine D"]
```

```
output = [85, 110, 95, 125]

plt.bar(machines, output)

plt.title("Machine Output Comparison")

plt.xlabel("Machines")

plt.ylabel("Output (units/hour)")

plt.show()
```

4. Line chart for monitoring engineering data

A **line chart** is useful when measurements change over time.

It can be used for:

- Temperature
- Pressure
- Voltage
- Current
- Vibration
- Speed
- Production rate
- Energy consumption

For example, an engineer can monitor the temperature of a machine over six hours.

```
import matplotlib.pyplot as plt

time = [1, 2, 3, 4, 5, 6]

temperature = [25, 28, 31, 35, 33, 30]

plt.plot(time, temperature, marker="o")

plt.title("Temperature vs Time")
```

```
plt.xlabel("Time (hours)")  
plt.ylabel("Temperature (°C)")  
plt.grid(True)  
plt.show()
```

The graph makes it easy to identify the **maximum temperature** and observe whether the temperature is increasing or decreasing.

5. Scatter plot for engineering relationships

A **scatter plot** is used to study the relationship between two numerical variables.

For example, an engineer can analyze the relationship between **load and deformation**.

```
import matplotlib.pyplot as plt  
load = [10, 20, 30, 40, 50]  
deformation = [2, 4, 6, 8, 10]  
plt.scatter(load, deformation)  
plt.title("Load vs Deformation")  
plt.xlabel("Load (kN)")  
plt.ylabel("Deformation (mm)")  
plt.show()
```

If the points form an approximately straight pattern, it indicates a strong relationship between load and deformation.

Scatter plots are useful for:

- Load vs deformation
- Speed vs fuel consumption
- Temperature vs pressure
- Voltage vs current

- Production time vs cost

6. Histogram for engineering data distribution

A **histogram** shows how numerical values are distributed.

For example, an engineer may want to analyze the distribution of manufactured component dimensions.

```
import matplotlib.pyplot as plt

diameter = [10.1, 10.2, 10.0, 10.3, 10.1,
            9.9, 10.2, 10.0, 10.1, 10.4]

plt.hist(diameter, bins=5)

plt.title("Component Diameter Distribution")

plt.xlabel("Diameter (mm)")

plt.ylabel("Frequency")

plt.show()
```

This helps engineers determine whether most components are close to the required dimension.

7. Box plot for detecting outliers

A **box plot** provides a compact summary of a dataset and can help identify unusual values.

For example, an engineer can analyze the vibration levels of a machine.

```
import matplotlib.pyplot as plt

vibration = [2.1, 2.3, 2.2, 2.4, 2.0, 2.3, 2.2, 5.8]

plt.boxplot(vibration)

plt.title("Machine Vibration Analysis")

plt.ylabel("Vibration Level")

plt.show()
```

The unusually high value may indicate an abnormal machine condition that requires investigation.

8. Heatmap for engineering analysis

A **heatmap** represents values using different levels of visual intensity. It is particularly useful for correlation analysis.

```
import seaborn as sns

import matplotlib.pyplot as plt

data = {

    "Temperature": [25, 30, 35, 40, 45],

    "Pressure": [10, 12, 15, 18, 20],

    "Efficiency": [95, 92, 88, 84, 80]

}

sns.heatmap(

    __import__("pandas").DataFrame(data).corr(),

    annot=True

)

plt.title("Engineering Parameter Correlation")

plt.show()
```

A heatmap can help identify which engineering parameters have strong positive or negative relationships.

9. Interactive visualization using Plotly

Plotly can be used to create interactive charts. Users can hover over points, zoom into regions, and explore data.

```
import plotly.express as px
```

```
time = [1, 2, 3, 4, 5]
temperature = [25, 28, 31, 35, 30]
fig = px.line(
    x=time,
    y=temperature,
    markers=True,
    title="Machine Temperature Monitoring"
)
fig.update_xaxes(title="Time (hours)")
fig.update_yaxes(title="Temperature (°C)")
fig.show()
```

Interactive visualization is especially useful in **engineering dashboards and monitoring systems**.

10. Important Python visualization libraries

Matplotlib

A general-purpose visualization library used for creating:

- Line charts
- Bar charts
- Scatter plots
- Histograms
- Pie charts
- 3D plots

Seaborn

Built on Matplotlib and useful for:

- Statistical visualization
- Correlation analysis
- Heatmaps
- Distribution plots
- Box plots

Plotly

Useful for:

- Interactive charts
- Engineering dashboards
- Web-based visualization
- Real-time data exploration

Pandas

Provides simple plotting capabilities directly from DataFrames.

```
import pandas as pd

data = {
    "Machine": ["A", "B", "C", "D"],
    "Temperature": [72, 80, 75, 90],
    "Pressure": [30, 35, 32, 40],
    "Output": [85, 110, 95, 125]
}

df = pd.DataFrame(data)

print(df)
```

Workflow Automation in Python

Workflow automation means using Python to **automatically perform repetitive tasks and processes** that would otherwise have to be done manually.

Instead of repeatedly doing the same steps, we write a Python program that performs them automatically, saving **time, effort, and reducing human errors**.

1. Why workflow automation is useful

Python automation helps to:

- Automate repetitive tasks
- Save time and effort
- Reduce human errors
- Process large amounts of data
- Organize files and folders
- Generate reports automatically
- Send emails automatically
- Extract information from websites
- Perform scheduled tasks

2. Common workflow automation tasks

Task	Python tool/library
Rename files	os, pathlib
Move/copy files	shutil
Read Excel files	pandas, openpyxl
Send emails	smtplib
Generate reports	pandas, openpyxl, reportlab
Web automation	Selenium, Playwright
Web requests	requests
Compress files	zipfile
Schedule tasks	schedule, cron
Database automation	sqlite3

3. Example: Automatically organize files

Suppose a folder contains:

Documents/

report.pdf

photo.jpg

data.xlsx

image.png

notes.txt

Instead of manually creating folders and moving files, Python can automatically organize them:

```
import os
import shutil
folder = "Documents"
for file in os.listdir(folder):
    file_path = os.path.join(folder, file)
    if os.path.isfile(file_path):
        extension = file.split(".")[-1].lower()
        if extension == "pdf":
            destination = "Documents/PDF"
        elif extension in ["jpg", "png", "jpeg"]:
            destination = "Documents/Images"
        elif extension in ["xlsx", "csv"]:
            destination = "Documents/Excel"
        elif extension == "txt":
            destination = "Documents/Text"
        else:
            continue
        os.makedirs(destination, exist_ok=True)
        shutil.move(file_path, destination)

print("Files organized successfully!")
```

4. How this workflow works

Start



Read files from folder



Identify file type



Create appropriate folder



Move file



Repeat for all files



Finish

5. Example using Pandas

Workflow automation can also be used for **data processing**.

For example, automatically calculate the total sales from an Excel/CSV file:

```
import pandas as pd
df = pd.read_csv("sales.csv")
df["Total"] = df["Quantity"] * df["Price"]
total_sales = df["Total"].sum()
print("Total Sales:", total_sales)
df.to_csv("processed_sales.csv", index=False)
```

Here Python automatically:

1. Reads the sales file.
2. Calculates the total for each product.
3. Calculates total sales.
4. Saves the processed data into a new file.

6. Real-world workflow example

A company receives daily sales data.

Without automation:

Download file



Open Excel



Calculate totals



Create report



Save report



Email report

With Python:

Python Script



Read sales data



Process data



Generate report



Save report



Send email automatically

This is especially useful when the same process needs to be performed **every day or every week**.

Analytics and AI Readiness – Preparing for Advanced Data Workflows

Analytics and AI readiness means preparing data, tools, systems, and programming skills so that they can be used effectively for **advanced analytics, machine learning, and artificial intelligence**.

Before building an AI model, the data must be properly collected, cleaned, analyzed, and prepared.

1. Why Analytics and AI Readiness is Important

It helps organizations and engineers to:

- Understand their data
- Find useful patterns and trends
- Clean poor-quality data
- Prepare data for machine learning
- Automate data-processing workflows
- Make data-driven decisions
- Build reliable AI applications
- Handle large datasets efficiently

2. Basic AI/Data Workflow

A typical advanced data workflow looks like this:

Data Collection



Data Cleaning



Data Transformation



Exploratory Data Analysis



Feature Engineering



Data Visualization



Machine Learning / AI



Model Evaluation



Deployment



Monitoring

3. Python Tools for AI Readiness

Purpose	Python Tool
Data manipulation	Pandas
Numerical calculations	NumPy
Visualization	Matplotlib, Seaborn
Machine Learning	Scikit-learn
Deep Learning	TensorFlow, PyTorch
Database access	SQLite, SQLAlchemy
Data formats	CSV, JSON, Excel
API integration	Requests
AI applications	AI/LLM APIs

4. Example Using Pandas

Suppose we have student data:

```
import pandas as pd

data = {
    "Name": ["Ravi", "Anu", "Kiran", "Meena"],
    "Marks": [85, 92, None, 78],
    "Attendance": [90, 95, 80, None]
}

df = pd.DataFrame(data)

print(df)
```

The data contains missing values. We can check them:

```
print(df.isnull().sum())
```

Output:

Name 0

Marks 1

Attendance 1

We can clean the missing values:

```
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
df["Attendance"] = df["Attendance"].fillna(df["Attendance"].mean())
print(df)
```

Now the dataset is more suitable for further analysis or machine learning.

5. Exploratory Data Analysis

After cleaning the data, we can analyze it:

```
print(df.describe())
```

This provides information such as:

- Mean
- Minimum
- Maximum
- Standard deviation
- Count

We can also calculate relationships between variables:

```
print(df.corr(numeric_only=True))
```

This can help identify whether variables such as **attendance and marks** are related.

6. Preparing Data for Machine Learning

For machine learning, we usually separate:

Features (X) → information used to make predictions

Target (y) → value we want to predict

Example:

```
X = df[["Attendance"]]
y = df["Marks"]
```

Then the data can be divided into training and testing datasets:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

The model can then learn from the training data.

7. Example of an AI-Ready Workflow

Suppose a company wants to predict customer purchases.

Customer Data



Collect data from database



Clean missing/incorrect values



Remove duplicate records



Analyze customer behavior



Create useful features



Split training/testing data



Train ML model



Evaluate model



Use model for predictions

Capstone Lab

Suppose a company has a file called:

sales.csv

It contains:

Date, Product, Quantity, Price

We want Python to automatically process this file.

Complete Workflow

Sales CSV File



Read Data



Check Missing/Duplicate Data



Clean Data



Calculate Total Sales



Analyze Products



Generate Charts



Create Report



Save Results



Finish

Step 1 – Read the Data

Using **Pandas**:

```
import pandas as pd
df = pd.read_csv("sales.csv")
print(df)
```

Step 2 – Clean the Data

Check missing values:

```
print(df.isnull().sum())
```

Remove duplicate records:

```
df = df.drop_duplicates()

Convert the date:

df["Date"] = pd.to_datetime(df["Date"])
```

Step 3 – Calculate Total Sales

Create a new column:

```
df["Total"] = df["Quantity"] * df["Price"]
print(df)
```

For example:

Product	Quantity	Price	Total
Laptop	2	50000	100000
Mouse	10	800	8000
Keyboard	5	1500	7500
Monitor	3	12000	36000

Calculate total revenue:

```
total_sales = df["Total"].sum()
print("Total Sales:", total_sales)
```

Step 4 – Perform Analytics

Find the best-selling products:

```
product_sales = df.groupby("Product")["Total"].sum()
print(product_sales)
```

Find the total quantity sold:

```
total_quantity = df["Quantity"].sum()
print("Total Quantity Sold:", total_quantity)
```

Find the average sale:

```
average_sale = df["Total"].mean()
print("Average Sale:", average_sale)
```

Step 5 – Generate Visualization

Using Matplotlib:

```
import matplotlib.pyplot as plt
product_sales.plot(kind="bar")
plt.title("Sales by Product")
plt.xlabel("Product")
plt.ylabel("Sales")
plt.tight_layout()
plt.savefig("sales_chart.png")
plt.close()
```

Python automatically creates a chart that can be included in the final report.

Step 6 – Save Processed Data

```
df.to_csv("processed_sales.csv", index=False)
```

Now the original data remains unchanged, while the cleaned data is saved separately.

Step 7 – Generate an Automated Report

A simple text report can be generated:

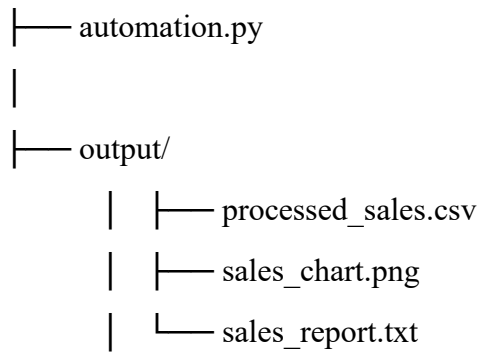
```
with open("sales_report.txt", "w") as file:
    file.write("SALES ANALYTICS REPORT\n")
    file.write("=====\n")
    file.write(f"Total Sales: {total_sales}\n")
    file.write(f"Total Quantity: {total_quantity}\n")
    file.write(f"Average Sale: {average_sale:.2f}\n")
print("Report generated successfully!")
```

Final Automated System

The complete project can be organized like this:

Sales Automation/

|
| — sales.csv



The user only needs to provide:

sales.csv

Python performs the rest automatically.

Real-World Capstone Workflow

A more advanced version can include:

